

INFORME

Mr. Sushi: Sistema de Pedidos

Cloud Computing

Integrantes del Grupo:

Choque Coaquira, Rafael Rodrigo	(Código: 202410378)
Borjas Bernaola, Gerald Marcelo Fernando	(Código: 202510059)
Huerta Roque, Francis Andres	(Código: 20231053)

Universidad de Ingeniería y Tecnología (UTEC)

2026

Resumen

El presente documento detalla la arquitectura de software implementada para la cadena de restaurantes “Mr. Sushi”. La solución ha sido diseñada bajo un modelo de arquitectura serverless nativa, orientada a eventos (EDA) e híbrida multi-nube.

El sistema gestiona de forma aislada las operaciones de 8 sedes físicas (comportándose como un esquema multi-tenant), mientras mantiene consolidados los datos de usuarios clientes globales y su programa de lealtad (Neki Puntos). La infraestructura backend corre principalmente sobre Amazon Web Services (AWS) utilizando el Serverless Framework para la orquestación del ciclo de vida, mientras que la integración con servicios de terceros (Rappi) se simula mediante una arquitectura en Google Cloud Platform (GCP) orquestada a través de Terraform.

Estructura General del Repositorio

La base de la solución completa se encuentra estructurada de manera modular en cuatro directorios raíz. El código fuente oficial, control de versiones e histórico de despliegues se encuentra disponible públicamente en el repositorio de GitHub:

github.com/gmborjasb/mr-sushi

- **mrsushi-backend/**: Contiene la totalidad de la lógica serverless en AWS, dividida en 7 microservicios independientes, cada uno provisto de su propia base de datos NoSQL y configuración de infraestructura en la nube.
- **frontend-trabajadores/**: Aplicación de Panel de Control SPA desarrollada sobre **React 19** y **Vite**, utilizada por cocineros, despachadores y repartidores.
- **mrsushi_clientes/**: Portal web transaccional público construido en **HTML5/CSS3/Vanilla JS** para los consumidores finales.
- **api-rappi-gcp/**: Código fuente e infraestructura como código (Terraform) de la API externa desplegada sobre Google Cloud Platform.

Arquitectura del Sistema

La arquitectura del sistema descansa sobre dos pilares tecnológicos principales: el desacoplamiento estricto a través de eventos y la orquestación asíncrona de flujos.

Diagrama de Arquitectura

A continuación, se detalla el diagrama lógico que describe el enrutamiento de peticiones síncronas (REST/HTTP) y la distribución asíncrona de eventos a través del bus personalizado:

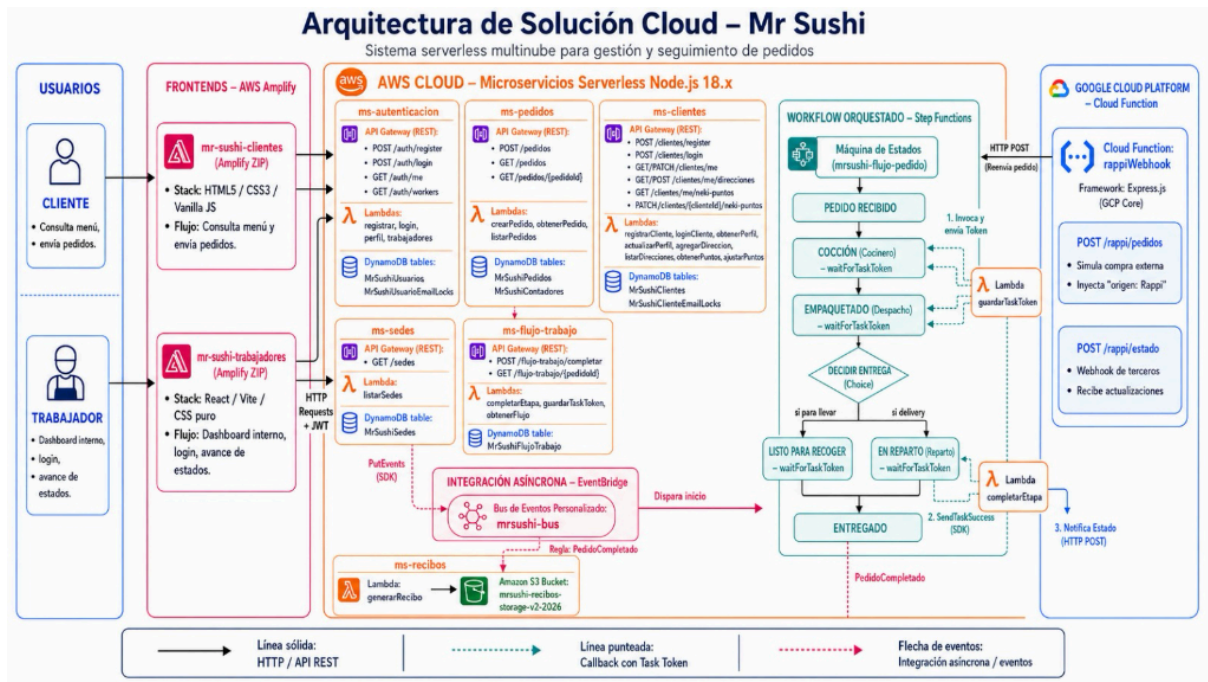


Figura 1: Diagrama de Arquitectura de Solución Híbrida (AWS + GCP)

Flujo de Orquestación y Estados (Diagrama de Secuencia)

El siguiente diagrama secuencial detalla cómo interactúan los componentes en la nube a lo largo del ciclo de vida de un pedido:

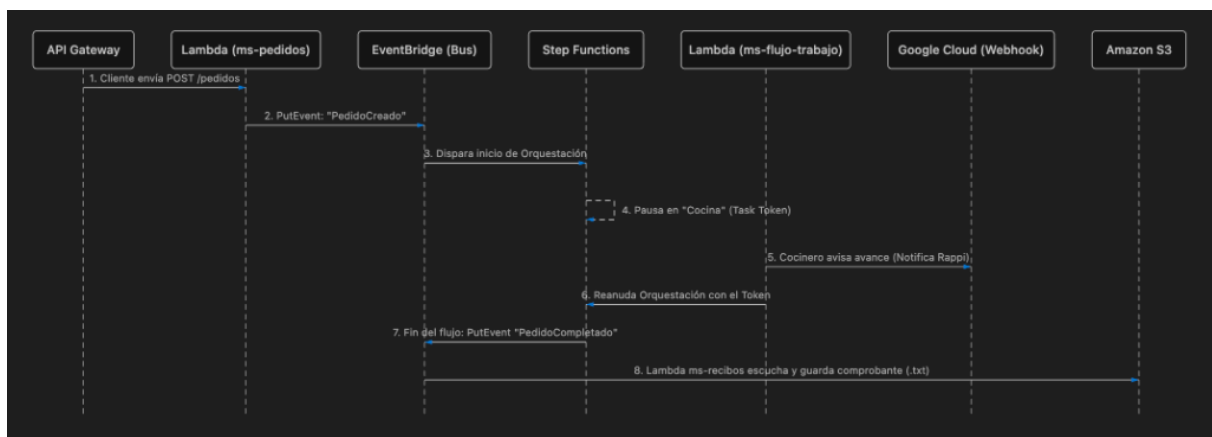


Figura 2: Secuencia asincrónica de orquestación de eventos y estados

Componentes y Orquestación en AWS

- **Amazon EventBridge (mrsushi-bus):** Actúa como el núcleo de mensajería (Event Bus). Al crearse un pedido, los microservicios publican eventos aislados. Ningún servicio llama a otro directamente; EventBridge intercepta el evento e inicia la orquestación.

- **AWS Step Functions:** Centraliza el flujo del pedido por medio de una máquina de estados reactiva. La máquina entra en estados de pausa (`waitForTaskToken`) para simular la interacción con el personal en la vida real.
- **AWS Lambda:** El motor de ejecución serverless del backend. Compuesto por funciones granulares en Node.js 18.x.
- **Amazon S3:** Destino final del flujo. Al completarse el ciclo de vida del pedido, la Lambda `generarRecibo` escribe en un bucket S3 un documento `.txt` inmutable que actúa como comprobante de pago.
- **AWS Amplify:** Plataforma de hosting serverless utilizada para alojar ambos frontends de forma independiente.

Desglose de Microservicios Backend

El backend se encuentra dividido en 7 microservicios completamente independientes. Cada servicio se despliega de manera aislada y tiene control absoluto sobre sus recursos de almacenamiento de datos:

Microservicio	Responsabilidad Técnica	Endpoints / Eventos
ms-sedes	Manejo y lectura de las coordenadas y radio de cobertura de las 8 sucursales físicas.	GET /sedes
ms-autenticacion	Registro, inicio de sesión y gestión de roles de trabajadores (JWT).	POST /auth/register POST /auth/login GET /auth/me
ms-clientes	Administración de perfiles de clientes, direcciones de entrega y monedero de Neki Puntos.	POST /clientes/register GET /clientes/ men PATCH /clientes/ me/neki-puntos
ms-pedidos	Recepción de órdenes, cálculo espacial de distancia a las sedes y emisión del evento <code>PedidoCreado</code> .	POST /pedidos GET /pedidos/{pedidoId}
ms-flujo-trabajo	Recibe las notificaciones de avance del personal y devuelve el Task Token al orquestador.	POST /flujo-trabajo/ completar
ms-stepfunctions	Definición e infraestructura CloudFormation de la Máquina de Estados del pedido.	Ejecución interna
ms-recibos	Escucha de forma asíncrona la finalización del pedido y escribe el recibo en S3.	Disparado por Event-Bridge

Flujo de Trabajo y Callback (Task Tokens)

La orquestación se gestiona de manera asíncrona mediante AWS Step Functions. Para evitar el consumo continuo de CPU en espera de la acción de los cocineros, despachadores y repartidores, se emplea el patrón **Wait for Callback con Task Token**:

1. Al iniciarse la máquina de estados, el flujo entra al estado `CoccionTask` invocando la integración `arn:aws:states:::lambda:invoke.waitForTaskToken`.
2. Step Functions detiene la ejecución del hilo y genera un identificador único en cadena llamado **Task Token**.
3. La Lambda asociada guarda este **Task Token** de forma segura en la tabla `MrSushiFlujoTrabajo` indexándolo por el `pedidoId`.
4. Cuando el trabajador presiona “Completar etapa” en el panel web, el Frontend hace un POST a la API Gateway de `ms-flujo-trabajo`.
5. La Lambda `completarEtapa` recupera el token de DynamoDB y ejecuta la llamada del SDK `SendTaskSuccessCommand` a la API de Step Functions. El flujo se reanuda de inmediato y pasa a la siguiente etapa de empaquetado.

Diseño de Base de Datos (DynamoDB)

El modelo de persistencia utiliza Amazon DynamoDB en modalidad bajo demanda (PAY_PER_REQUEST). Para cumplir con la flexibilidad del diseño serverless, se utilizan llaves descriptivas en lugar de IDs genéricos:

- **MrSushiSedes**: Almacena información estática de las 8 sedes. Llave: `sedeId` (HASH).
- **MrSushiUsuarios**: Registra credenciales de trabajadores. Llaves: `sedeId` (HASH) y `email` (RANGE).
- **MrSushiUsuarioEmailLocks**: Control transaccional (`TransactWriteItems`) para asegurar que no se registren correos duplicados en distintas sedes. Llave: `email` (HASH).
- **MrSushiPedidos**: Información de pedidos. Llaves: `sedeId` (HASH) y `pedidoId` (RANGE). Provee índices secundarios globales (GSI) `ClienteIndex` y `SedeCreatedIndex` para consultas veloces.
- **MrSushiContadores**: Mantiene contadores diarios por sede para generar números de ticket secuenciales legibles (ej: ticket 1, 2 del día).

Integración Multi-Cloud (GCP)

Para cumplir los requerimientos multi-nube, se desacopló el envío de notificaciones hacia el sistema de delivery Rappi.

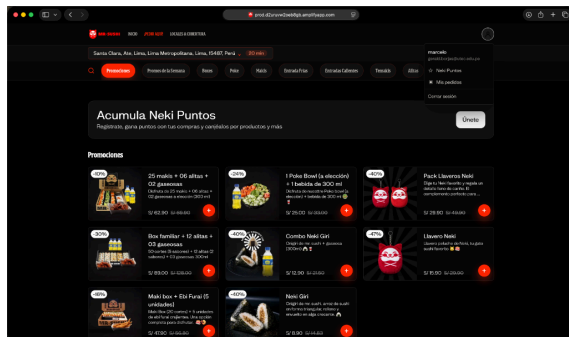
- La infraestructura de Google Cloud se levantó de manera declarativa con **Terraform**.
- Se desplegó una **Google Cloud Function** escrita sobre Express.js (`rappiWebhook`).

- Cuando un pedido cambia de estado en AWS, el Lambda `completarEtapa` envía un HTTP POST al endpoint de actualización de GCP (`/rappi/estado`) simulando la pantalla del motorizado de Rappi.
- El simulador de Rappi también puede inyectar pedidos de forma inversa llamando síncronamente al endpoint de creación de AWS a través de la ruta `/rappi/pedidos`.

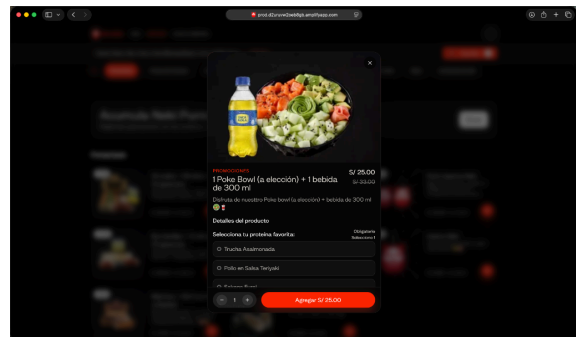
Evidencias de Funcionamiento

A continuación, se presentan las capturas de pantalla tomadas directamente de los servicios en ejecución como evidencia del funcionamiento de la plataforma:

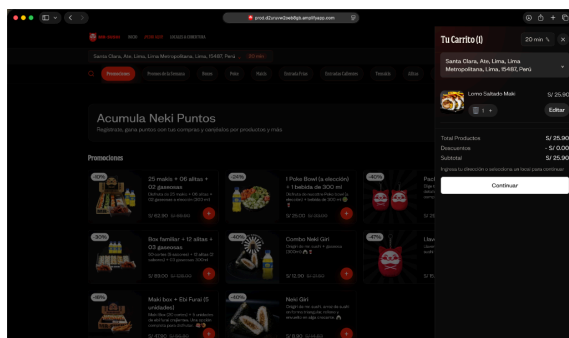
Portal Transaccional de Clientes



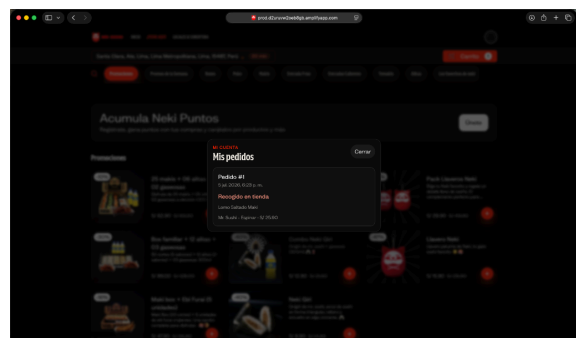
Vista de inicio y catálogo de productos



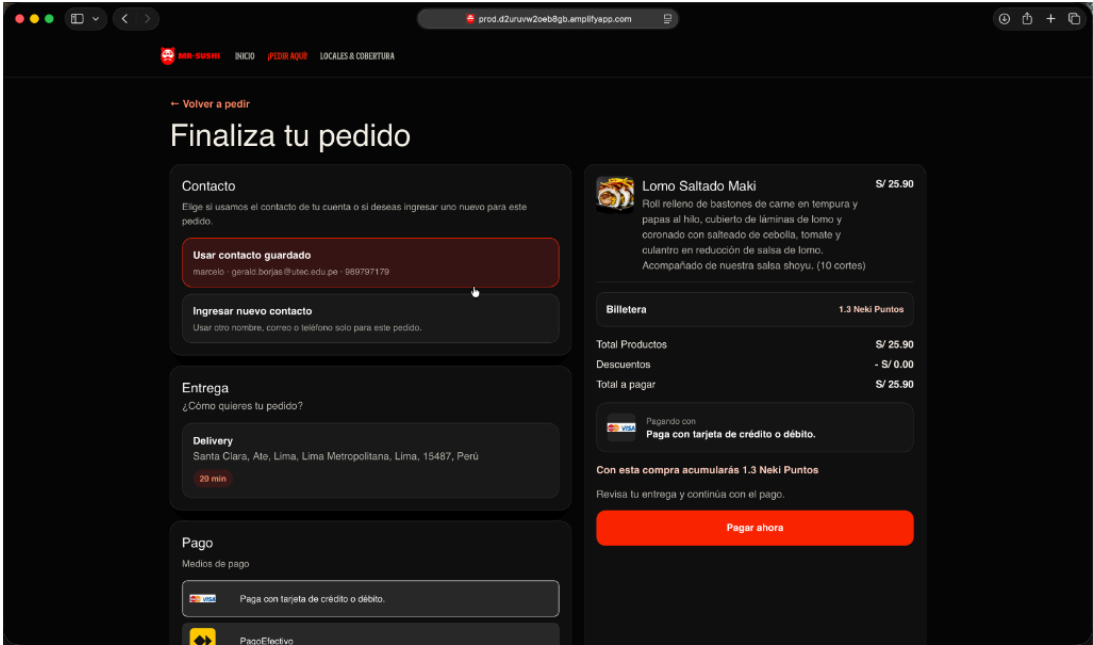
Detalle de producto y agregados



Flujo lateral del carrito de compras

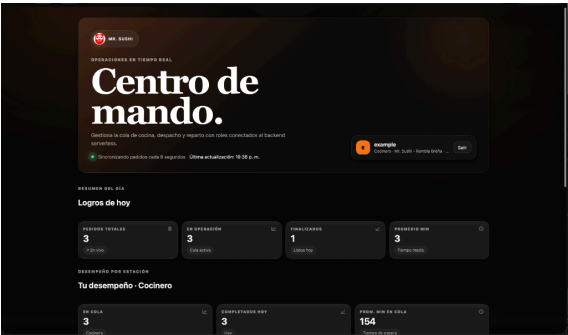


Historial de compras del cliente

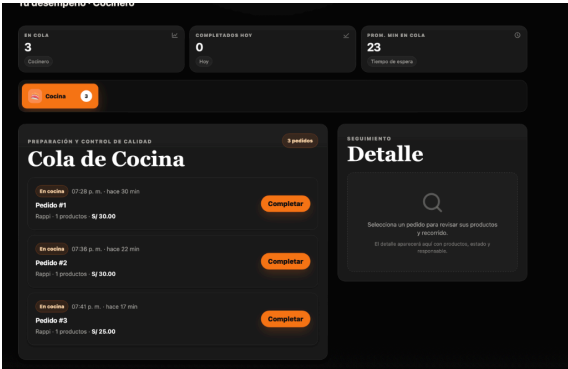


Checkout final, selección de sede de entrega y cálculo de Neki Puntos

Panel de Control de Trabajadores (React / Vite)

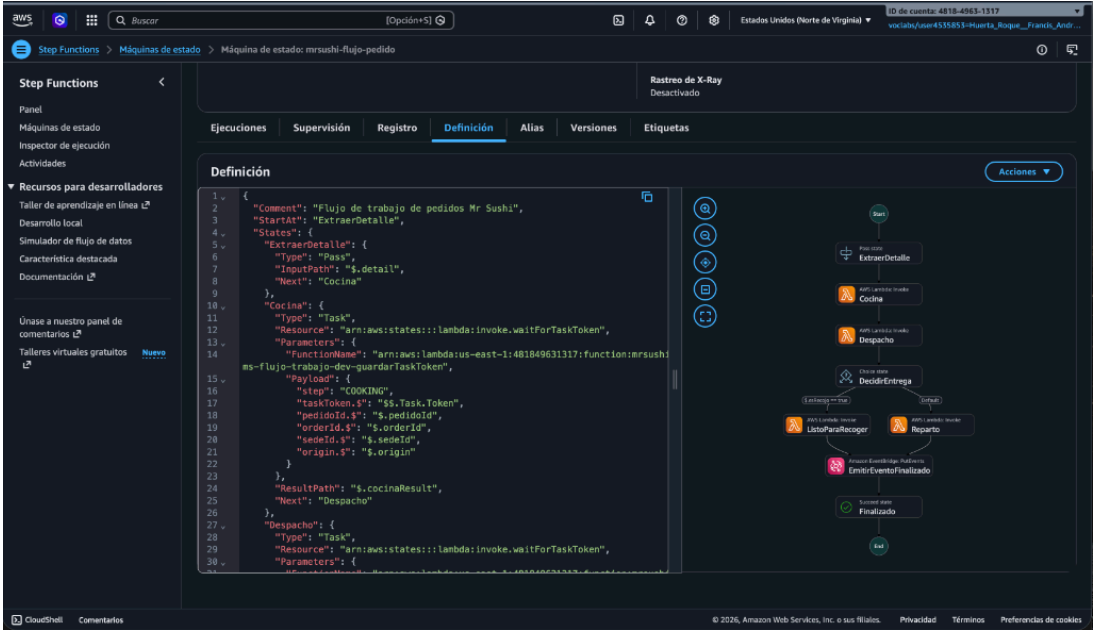


Centro de mando y resumen métrico diario



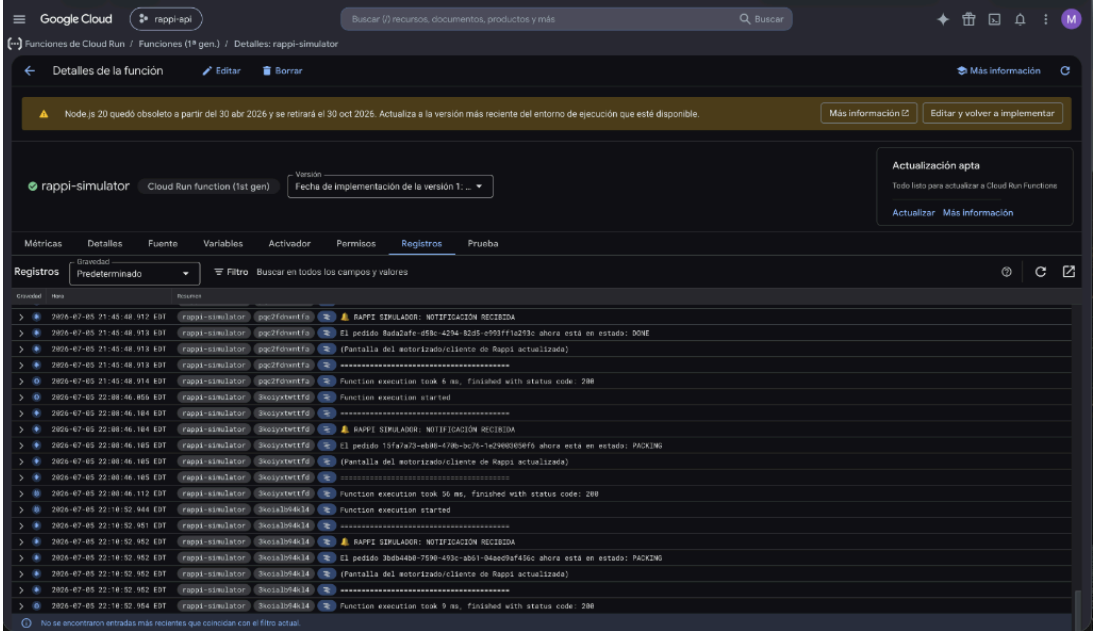
Cola de pedidos activa en Cocina

Orquestación y Máquina de Estados (AWS Step Functions)



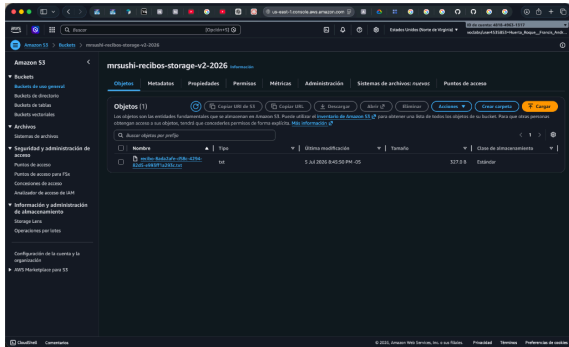
Grafo de definición y estados del flujo del pedido en Step Functions (Consola AWS)

Integración y Logs en Google Cloud Platform (GCP Webhook)

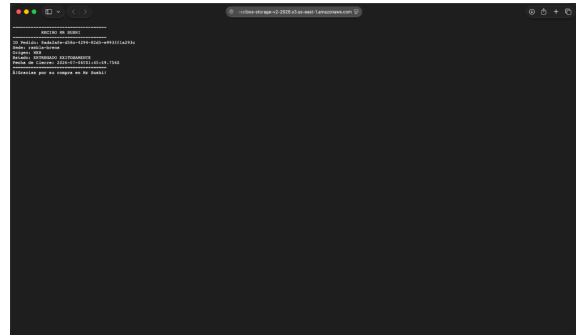


Logs en tiempo real de Google Cloud Run Functions recibiendo y validando las peticiones POST de AWS

Almacenamiento de Comprobantes (.txt) en Amazon S3

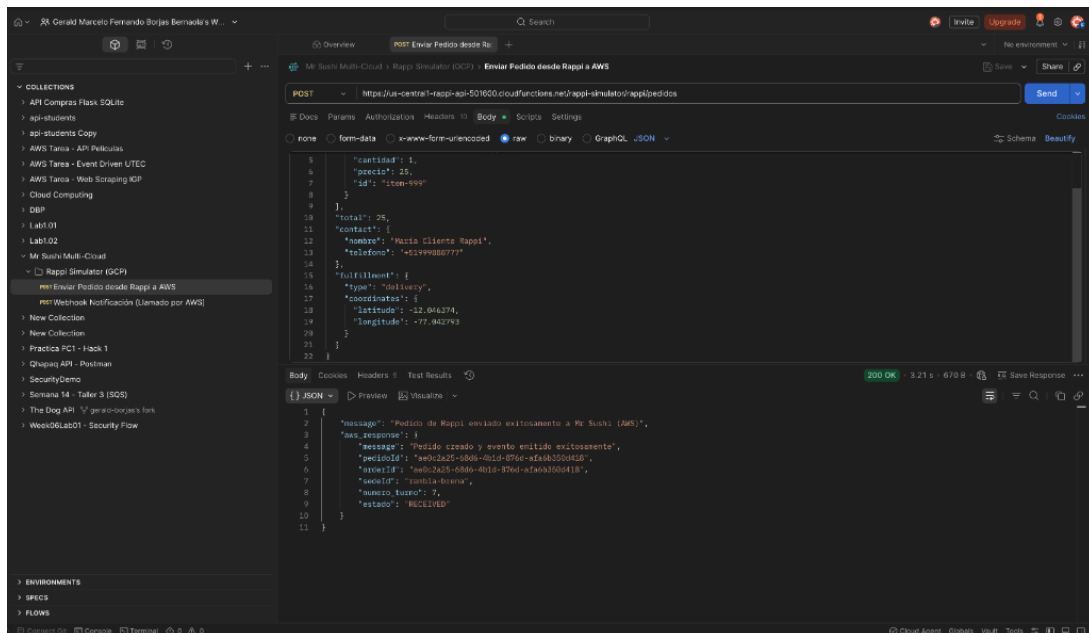


Registro de recibos guardados en el bucket S3



Detalle y contenido del archivo de recibo generado

Integración y Pruebas con Postman



Ejecución de llamadas HTTP y pruebas de la colección del simulador de Rappi en Postman

Conclusiones y Limitaciones

1. **Escalabilidad Serverless:** Toda la infraestructura backend se ajusta de manera elástica según la concurrencia de clientes. Al no contar con servidores aprovisionados en estado ocioso, el costo operativo de la plataforma se aproxima a cero en horas de baja demanda.
2. **Limitaciones del Learner Lab:** La infraestructura corre sobre las cuentas AWS Academy Learner Labs, cuyas credenciales expiran y se reciclan periódicamente. Por tanto, los recursos (DynamoDB, Lambdas, S3) son de naturaleza temporal, aunque la lógica del código y la arquitectura están diseñadas para producción continua en un entorno corporativo real de AWS.

3. **Seguridad y Aislamiento:** El aislamiento multi-sede se resolvió a nivel lógico encriptando la sucursal del trabajador dentro del token JWT de sesión. Así se evita la manipulación maliciosa de IDs de sedes por parte del navegador cliente.